

1. **Why can't a struct inherit from another struct or class in C#?**

Structs are value types and implicitly sealed, so they cannot inherit from other structs or classes. They can, however, implement interfaces.

2. **How do access modifiers impact the scope and visibility of a class member?**

Access modifiers control where a member can be accessed:

- a. private: only within the containing class.
- b. internal: anywhere within the same assembly/project.
- c. public: from any accessible code.

3. **Why is encapsulation critical in software design?**

Encapsulation protects an object's internal data and allows controlled access through methods or properties. It improves security, maintainability, and prevents invalid state changes.

4. **What are constructors in structs?**

Constructors initialize struct instances. Structs have a default constructor that initializes fields to their default values and can also have parameterized constructors. Modern C# allows explicit parameterless struct constructors as well.

5. **How does overriding methods like ToString() improve code readability?**

It provides a meaningful textual representation of an object, making debugging, logging, and displaying values clearer and easier to understand.

6. **How does memory allocation differ for structs and classes in C#?**

Structs are value types and store their data directly wherever the variable is located. Classes are reference types; their objects are usually allocated on the heap, while variables store references to those objects. Structs are copied by value, whereas class references are copied